Lookup 최적화 (Lookup Optimization)

개요

Lookup은 ZKEVM 회로에서 가장 비용이 많이 드는 연산 중 하나입니다. 효율적인 lookup 최적화는 회로 크기, 증명 생성 시간, 메모리 사용량에 직접적인 영향을 미칩니다. 이 문서에서는 ZKEVM 구현체에서 사용되는 다양한 lookup 최적화 기법을 설명합니다.

Lookup 비용 요소

Lookup의 비용은 다음 요소들로 구성됩니다:

1. **테이블 크기**: 테이블의 행 수 (증명 크기에 비례)
2. **Lookup 횟수**: 각 lookup은 제약 조건 추가 (회로 복잡도 증가)
3. **컬럼 수**: 다중 컬럼 lookup은 RLC 계산 필요 (계산 비용 증가)
4. **Phase**: SecondPhase 컬럼은 FirstPhase보다 증명 생성 느림

최적화 목표:

- 테이블 크기 최소화
- 불필요한 lookup 제거
- 컬럼 수 감소
- 적절한 Phase 선택

1. 테이블 크기 최적화

1.1 중복 제거

테이블에 중복된 데이터를 저장하지 않아 크기를 줄입니다.

예시: Bytecode Table

파일: `zkevm-circuits/src/bytecode_circuit.rs`

```
// 코드 해시로 중복 제거
pub fn assign_bytecode_table(
&self,
layouter: &mut impl Layouter<F>,
bytecodes: &[Bytecode],
challenges: &Challenges<Value<F>>,
) -> Result<(), Error> {
// 중복된 코드 해시 제거
let unique_bytecodes: BTreeMap<H256, &Bytecode> = bytecodes
.iter()
.map(|b| (b.hash(), b))
.collect();
layouter.assign_region(
|| "bytecode table",
|mut region| {
let mut offset = 0;
// 고유한 바이트코드만 테이블에 할당
for bytecode in unique_bytecodes.values() {
for (index, byte) in bytecode.code.iter().enumerate() {
```

```
// 각 바이트를 한 행에 할당
region.assign_advice(
|| "bytecode byte",
self.bytecode_table.value,
offset,
|| Value::known(F::from(*byte  as u64)),
)?;
offset += 1;
}
}
Ok(())
},
)
}
```

효과:

- 같은 바이트코드가 여러 트랜잭션에서 사용되어도 한 번만 저장
- 테이블 크기 대폭 감소 (특히 인기 있는 컨트랙트의 경우)
- 메모리 사용량 감소

1.2 효율적인 인코딩

데이터를 압축하여 테이블 크기를 줄입니다.

예시: Fixed Table 최적화

```rust
// ❌ 비효율적: 모든 조합 저장
// U8 × U8 → U16 테이블: 256 × 256 = 65,536 행

// ✅ 효율적: 필요한 값만 저장
pub struct U8Table {
pub value: Column<Fixed>,
}

impl U8Table {
pub fn load(&self, layouter: &mut impl Layouter<F>) -> Result<(), Error> {
layouter.assign_region(
|| "u8 table",
|mut region| {
// 0~255만 저장: 256 행
for i in 0..256 {
region.assign_fixed(
|| format!("u8[{}]", i),
self.value,
i,
|| Value::known(F::from(i as u64)),
)?;
}
```

```
Ok(())
},
)
}
}
```

최적화 전략:

- 범위 테이블: 필요한 범위만 저장 (예: U8, U10, U16)
- 비트 연산 테이블: 자주 사용되는 연산만 포함
- 조건부 테이블: 특정 조건에서만 활성화

1.3 테이블 분할

큰 테이블을 여러 작은 테이블로 분할하여 효율성을 높입니다.

예시: RW Table 분할 전략

```
// 개념적 분할 (실제 구현은 단일 테이블)
//
// RW Table을 논리적으로 분할:
// 1. Stack 연산 (tag = 2)
// 2. Memory 연산 (tag = 3)
// 3. Storage 연산 (tag = 4)
// 4. Account 연산 (tag = 5-9)
// 5. TxAccessList 연산 (tag = 10-11)
```

```
// Lookup 시 tag로 필터링
pub(crate) fn  stack_lookup(
&mut  self,
is_write: Expression<F>,
stack_pointer_offset: Expression<F>,
value: Expression<F>,
) {
let  input_expr = rlc::expr(
&[
self.curr.state.rw_counter.expr() + self.rw_counter_offset.clone(),
is_write,
RwTableTag::Stack.expr(), // tag = 2로 필터링
self.curr.state.call_id.expr(),
self.curr.state.stack_pointer.expr() + stack_pointer_offset,
0.expr(), // field_tag
0.expr(), // storage_key
value,
0.expr(), // value_prev
0.expr(), // aux1
0.expr(), // aux2
],
self.challenges.lookup_input(),
);
```

```
self.add_lookup("stack lookup", vec![(input_expr, table_expr)]);
}
```

장점:

- 각 lookup이 테이블의 일부만 검색
- 캐시 효율성 향상
- 병렬 처리 가능성

1.4 패딩 최소화

테이블 패딩을 최소화하여 불필요한 행을 줄입니다.

파일: `zkevm-circuits/src/table.rs`

```
// 조건부 lookup을 위한 첫 행 패딩
pub fn load_with_padding<F: Field>(
&self,
layouter: &mut impl Layouter<F>,
data: &[Entry],
) -> Result<(), Error> {
layouter.assign_region(
|| "table with padding",
|mut region| {
// 첫 행만 0으로 패딩 (조건부 lookup용)
region.assign_fixed(
|| "q_enable padding",
```

```
self.q_enable,
0,
|| Value::known(F::zero()),
)?;
// 나머지 컬럼도 0으로 초기화
for  column  in  self.advice_columns() {
region.assign_advice(
|| "padding",
column,
0,
|| Value::known(F::zero()),
)?;
}
// 실제 데이터는 offset 1부터 시작
for (i, entry) in  data.iter().enumerate() {
let  offset = i + 1;
region.assign_fixed(
|| "q_enable",
self.q_enable,
offset,
|| Value::known(F::one()),
)?;
// 데이터 할당...
```

```
}
Ok(())
},
)
}
```

최적화 효과:

- 패딩 행: 1개만 필요 (조건부 lookup용)
- 테이블 끝 패딩: 불필요 (Halo2가 자동 처리)
- 메모리 효율성 향상

2. Lookup 횟수 감소

2.1 조건부 Lookup

불필요한 lookup을 조건부로 제거합니다.

예시: PUSH 명령어 처리

파일: `bus-mapping/src/evm/opcodes/push.rs`

```
// PUSH 명령어일 때만 다음 바이트들을 lookup
impl<F: Field> ExecutionGadget<F> for PushGadget<F> {
fn  configure(cb: &mut ConstraintBuilder<F>) -> Self {
let  opcode = cb.query_cell();
// PUSH 크기 계산 (PUSH1=1, PUSH2=2, ..., PUSH32=32)
let  push_size = opcode.expr() - OpcodeId::PUSH1.expr() + 1.expr();
```

```
// PUSH 데이터 바이트들
let push_data = (0..32)
.map(|_| cb.query_cell())
.collect::<Vec<_>>();
// 조건부 lookup: PUSH 크기만큼만 lookup 수행
for i in 0..32 {
let is_push_byte = i.expr().lt(push_size.clone());
cb.condition(is_push_byte, |cb| {
cb.bytecode_lookup(
cb.curr.state.code_hash.expr(),
cb.curr.state.program_counter.expr() + i.expr() + 1.expr(),
0.expr(), // is_code = false
push_data[i].expr(),
);
});
}
Self { opcode, push_data, /* ... */ }
}
}
```

효과:

- PUSH1: 1번 lookup (31번 절약)
- PUSH2: 2번 lookup (30번 절약)

- PUSH32: 32번 lookup (절약 없음)
- 평균적으로 lookup 횟수 대폭 감소

2.2 Lookup 통합

여러 개의 작은 lookup을 하나의 큰 lookup으로 통합합니다.

예시: 다중 바이트 검증

```
// ❌ 비효율적: 각 바이트마다 개별 lookup
for byte in bytes.iter() {
cb.lookup("byte range check", vec![
(byte.expr(), u8_table.value.expr())
]);
}
// 총 lookup 횟수: n번 (n = 바이트 수)

// ✅ 효율적: RLC로 통합하여 한 번에 lookup
let bytes_rlc = rlc::expr(&bytes, challenges.keccak_input());
cb.lookup("bytes range check", vec![
(bytes_rlc, bytes_table_rlc.expr())
]);
// 총 lookup 횟수: 1번
```

Trade-off:

- **장점**: Lookup 횟수 감소, 제약 조건 감소

- **단점**: RLC 계산 비용, Phase 2 컬럼 필요
- **적용 기준**: 3개 이상의 값을 lookup할 때 유리

2.3 Lookup 캐싱

같은 값을 여러 번 lookup하는 경우 결과를 캐싱합니다.

예시: 반복되는 Bytecode Lookup

```
// 같은 opcode를 여러 ExecutionGadget에서 사용
pub struct OpcodeCache<F> {
// (code_hash, pc) → opcode 매핑
cache: HashMap<(Expression<F>, Expression<F>), Cell<F>>,
}

impl<F: Field> OpcodeCache<F> {
// 첫 번째 사용: lookup 수행 및 캐싱
pub fn get_or_lookup(
&mut self,
cb: &mut ConstraintBuilder<F>,
code_hash: Expression<F>,
pc: Expression<F>,
) -> Cell<F> {
let key = (code_hash.clone(), pc.clone());
if let Some(cached) = self.cache.get(&key) {
// 캐시된 값 반환 (lookup 불필요)
```

```
cached.clone()

} else {

// 새로운 셀 할당 및 lookup

let  opcode = cb.query_cell();

cb.bytecode_lookup(

code_hash,

pc,

1.expr(), // is_code

opcode.expr(),

);

self.cache.insert(key, opcode.clone());

opcode

}

}

}
```

적용 사례:

- 같은 PC의 opcode를 여러 gadget에서 참조
- 반복되는 스토리지 읽기
- 공통 상수 값 lookup

2.4 Selector 기반 최적화

Selector를 사용하여 특정 행에서만 lookup을 활성화합니다.

파일: zkevm-circuits/src/evm_circuit/execution.rs

```rust
// ExecutionState별로 필요한 lookup만 수행
pub struct ExecutionConfig<F> {
// 각 ExecutionState에 대한 selector
q_step: Selector,
q_step_first: Selector,
q_step_last: Selector,
// ExecutionState별 gadget
add_gadget: Box<AddGadget<F>>,
mul_gadget: Box<MulGadget<F>>,
// ...
}

impl<F: Field> ExecutionConfig<F> {
pub fn configure(
meta: &mut ConstraintSystem<F>,
challenges: Challenges<Expression<F>>,
// ...
) -> Self {
// ADD gadget: 스택 lookup만 필요
let add_gadget = AddGadget::configure(cb);
// SLOAD gadget: 스택 + 스토리지 lookup 필요
```

```
let  sload_gadget = SloadGadget::configure(cb);

// 각 gadget은 자신의 selector가 활성화될 때만 lookup 수행

Self {

q_step,

add_gadget: Box::new(add_gadget),

sload_gadget: Box::new(sload_gadget),

// ...

}

}

}
```

효과:

- 각 step에서 필요한 lookup만 활성화
- 불필요한 제약 조건 제거
- 회로 크기 감소

3. 컬럼 수 조정

3.1 RLC를 통한 컬럼 압축

여러 컬럼을 RLC로 압축하여 단일 lookup으로 변환합니다.

예시: Keccak Table Lookup

```
// 원본: 5개 컬럼 lookup

// - is_enabled
```

```
// - is_final
// - input_rlc
// - input_len
// - output_rlc


// RLC 압축 전
meta.lookup_any("keccak lookup (5 columns)", |meta| {
vec![
(is_enabled.expr(), table.is_enabled.expr()),
(is_final.expr(), table.is_final.expr()),
(input_rlc.expr(), table.input_rlc.expr()),
(input_len.expr(), table.input_len.expr()),
(output_rlc.expr(), table.output_rlc.expr()),
]
});
// 제약 조건: 5개


// RLC 압축 후
meta.lookup_any("keccak lookup (1 RLC)", |meta| {
let  input_expr = rlc::expr(
&[
is_enabled.expr(),
is_final.expr(),
```

```
input_rlc.expr(),
input_len.expr(),
output_rlc.expr(),
],
challenges.lookup_input(),
);
let  table_expr = rlc::expr(
&[
table.is_enabled.expr(),
table.is_final.expr(),
table.input_rlc.expr(),
table.input_len.expr(),
table.output_rlc.expr(),
],
challenges.lookup_input(),
);
vec![(input_expr, table_expr)]
});
// 제약 조건: 1개 + RLC 계산 비용
```

비용 분석:

- **RLC 없이**: 5개 lookup 제약 조건
- **RLC 사용**: 1개 lookup + 5개 RLC 계산

- **순이익**: 제약 조건 감소, 증명 크기 감소

3.2 컬럼 재사용

같은 컬럼을 여러 용도로 재사용하여 컬럼 수를 줄입니다.

파일: `zkevm-circuits/src/evm_circuit/util.rs`

```
// CellManager의 컬럼 타입 분류
pub(crate) enum CellType {
StoragePhase1, // 일반 저장소
StoragePhase2, // Phase 2 저장소
StoragePermutation, // 복사 제약 가능
StoragePermutationPhase2, // Phase 2 복사 제약
Lookup(Table), // 특정 테이블 lookup 전용
LookupByte, // 바이트 lookup 전용
}

impl<F: Field> CellManager<F> {
// Storage 요청 시 Permutation 컬럼도 사용 가능
fn next_column(&self, cell_type: CellType) -> usize {
let mut best_index: Option<usize> = None;
let mut best_height = self.height;
// 요청된 타입의 컬럼 찾기
for column in self.columns.iter() {
if column.cell_type == cell_type && column.height < best_height {
```

```
best_index = Some(column.index);
best_height = column.height;
}
}
// StoragePhase1 요청 시 StoragePermutation도 사용 가능
if cell_type == CellType::StoragePhase1 {
for column in self.columns.iter() {
if column.cell_type == CellType::StoragePermutation
&& column.height < best_height {
best_index = Some(column.index);
best_height = column.height;
}
}
}
best_index.expect("not enough cells")
}
}
```

재사용 전략:

- **Permutation 컬럼**: 복사 제약이 필요 없을 때 일반 저장소로 사용
- **Lookup 컬럼**: 특정 테이블 lookup 전용이지만 여유 공간은 다른 용도로 활용
- **Phase 컬럼**: Phase 1 데이터는 Phase 2 컬럼에도 저장 가능

3.3 Lookup 전용 컬럼 할당

자주 사용되는 lookup에는 전용 컬럼을 할당하여 효율성을 높입니다.

파일: `zkevm-circuits/src/evm_circuit/param.rs`

```rust
// Lookup 설정: (테이블, 전용 컬럼 수)
pub(crate) const LOOKUP_CONFIG: &[(Table, usize)] = &[
(Table::Fixed, 8), // Fixed 테이블: 8개 컬럼
(Table::Tx, 4), // Tx 테이블: 4개 컬럼
(Table::Rw, 8), // RW 테이블: 8개 컬럼
(Table::Bytecode, 4), // Bytecode 테이블: 4개 컬럼
(Table::Block, 1), // Block 테이블: 1개 컬럼
(Table::Copy, 1), // Copy 테이블: 1개 컬럼
(Table::Keccak, 1), // Keccak 테이블: 1개 컬럼
(Table::Exp, 1), // Exp 테이블: 1개 컬럼
];

// 총 lookup 전용 컬럼: 28개
pub(crate) const N_BYTE_LOOKUPS: usize = 28;
```

할당 전략:

- **자주 사용**: 더 많은 전용 컬럼 (Fixed: 8개, RW: 8개)
- **가끔 사용**: 적은 전용 컬럼 (Block: 1개, Keccak: 1개)
- **균형**: 컬럼 수와 lookup 빈도의 균형 유지

효과:

- Lookup 병렬 처리 가능

- 컬럼 경합 감소
- 회로 높이 최소화

3.4 동적 컬럼 할당

필요에 따라 컬럼을 동적으로 할당합니다.

파일: `zkevm-circuits/src/keccak_circuit/cell_manager.rs`

```
// Keccak Circuit의 동적 컬럼 생성
pub(crate) fn query_cell_at_pos(
&mut self,
meta: &mut ConstraintSystem<F>,
row_idx: i32,
column_idx: usize,
) -> Cell<F> {
let column = if column_idx < self.columns.len() {
// 기존 컬럼 재사용
self.columns[column_idx].advice
} else {
// 새 컬럼 생성
let advice = meta.advice_column();
let mut expr = 0.expr();
meta.create_gate("Query column", |meta| {
expr = meta.query_advice(advice, Rotation::cur());
vec![0.expr()]
```

```
});
self.columns.push(CellColumn { advice, expr });
advice
};


// 셀 생성 및 반환
let mut cells = Vec::new();
meta.create_gate("Query cell", |meta| {
cells.push(Cell::new(meta, column, column_idx, row_idx));
vec![0.expr()]
});
cells[0].clone()
}
```

장점:

- 실제 필요한 만큼만 컬럼 생성
- 메모리 효율성
- 유연한 회로 구조

단점:

- 컴파일 타임에 크기 결정 불가
- 동적 할당 오버헤드

4. Phase 최적화

4.1 Phase별 컬럼 분리

적절한 Phase에 컬럼을 배치하여 증명 생성 시간을 최적화합니다.

Phase 구조:

```
FirstPhase (Phase 0)
↓
- Fixed 컬럼
- 기본 Advice 컬럼
- Challenge 없는 제약 조건
↓
SecondPhase (Phase 1)
↓
- RLC 계산 컬럼
- evm_word, keccak_input challenge 사용
- RLC 기반 제약 조건
↓
ThirdPhase (Phase 2) - SuperCircuit만
↓
- Lookup RLC 컬럼
- lookup_input challenge 사용
- 최종 lookup 제약 조건
```

최적화 원칙:

```rust
// ✅ 좋은 예: Phase 1 데이터는 Phase 1 컬럼에
let basic_data = meta.advice_column(); // FirstPhase
meta.create_gate("basic constraint", |meta| {
let data = meta.query_advice(basic_data, Rotation::cur());
vec![data * (data - 1.expr())] // Boolean 제약
});

// ✅ 좋은 예: RLC 데이터는 Phase 2 컬럼에
let rlc_data = meta.advice_column_in(SecondPhase);
meta.create_gate("rlc constraint", |meta| {
let rlc = meta.query_advice(rlc_data, Rotation::cur());
let challenge = meta.query_challenge(challenges.evm_word);
vec![rlc - challenge]
});

// ❌ 나쁜 예: 모든 데이터를 Phase 2에
let unnecessary_phase2 = meta.advice_column_in(SecondPhase);
// Challenge를 사용하지 않는데도 Phase 2 사용 → 비효율적
```

4.2 Phase 2 컬럼 최소화

Phase 2 컬럼은 증명 생성이 느리므로 최소한으로 유지합니다.

파일: `zkevm-circuits/src/evm_circuit/param.rs`

```
// Phase 2 컬럼 설정
pub(crate) const N_PHASE2_COLUMNS: usize = 4;
pub(crate) const N_PHASE2_COPY_COLUMNS: usize = 1;


// 총 Phase 2 컬럼: 5개 (전체 컬럼의 약 5%)
```

사용 전략:

- **RLC 저장**: Word RLC, Keccak RLC 등
- **복사 제약**: Phase 2 데이터 간 equality
- **최소화**: 꼭 필요한 경우만 사용

비용 비교:

```
Phase 1 컬럼: 기준 시간 1x
Phase 2 컬럼: 약 1.5x ~ 2x 시간
Phase 3 컬럼: 약 2x ~ 3x 시간 (SuperCircuit)
```

4.3 Challenge 재사용

같은 Phase의 challenge를 재사용하여 Phase 수를 줄입니다.

파일: `zkevm-circuits/src/util.rs`

```
// 3개의 challenge를 2개 Phase에 분산
pub  struct Challenges<T = Challenge> {
evm_word: T, // SecondPhase
```

```
keccak_input: T, // SecondPhase
lookup_input: Option<T>, // ThirdPhase (SuperCircuit만)
}

impl Challenges {
// 개별 회로: Phase 2까지만 사용
pub fn construct_p1<F: Field>(meta: &mut ConstraintSystem<F>) -> Self {
Self {
evm_word: meta.challenge_usable_after(FirstPhase),
keccak_input: meta.challenge_usable_after(FirstPhase),
lookup_input: None, // Phase 3 불필요
}
}

// SuperCircuit: Phase 3까지 사용
pub fn construct<F: Field>(meta: &mut ConstraintSystem<F>) -> Self {
Self {
evm_word: meta.challenge_usable_after(FirstPhase),
keccak_input: meta.challenge_usable_after(FirstPhase),
lookup_input: Some(meta.challenge_usable_after(SecondPhase)),
}
}
```

```
}
```

최적화 효과:

- 개별 회로: 2 Phase (빠른 증명 생성)
- SuperCircuit: 3 Phase (회로 간 lookup 지원)
- Phase 수 최소화로 전체 증명 시간 단축

5. 실전 최적화 패턴

5.1 Batch Lookup

여러 lookup을 배치로 처리하여 효율성을 높입니다.

```
// ❌ 비효율적: 개별 lookup
for i in 0..n {
cb.stack_lookup(
false.expr(), // is_write
i.expr(), // offset
values[i].expr(),
);
}
// n번의 lookup 호출


// ✅ 효율적: 배치 lookup
let lookups: Vec<_> = (0..n)
```

```
.map(|i| {
let  input_expr = rlc::expr(
&[
rw_counter.expr() + i.expr(),
0.expr(), // is_write
RwTableTag::Stack.expr(),
call_id.expr(),
stack_pointer.expr() + i.expr(),
0.expr(), 0.expr(),
values[i].expr(),
0.expr(), 0.expr(), 0.expr(),
],
challenges.lookup_input(),
);
(input_expr, table_expr.clone())
})
.collect();


cb.add_lookups("batch stack lookup", lookups);
// 1번의 lookup 호출로 n개 처리
```

효과:

- 함수 호출 오버헤드 감소

- 컴파일러 최적화 기회 증가
- 코드 가독성 향상

5.2 Lazy Lookup

실제로 필요할 때만 lookup을 수행합니다.

```
// 조건부 실행 gadget
impl<F: Field> ExecutionGadget<F> for ConditionalGadget<F> {
fn  configure(cb: &mut ConstraintBuilder<F>) -> Self {
let  condition = cb.query_cell();
let  value = cb.query_cell();
// condition이 true일 때만 lookup
cb.condition(condition.expr(), |cb| {
cb.stack_lookup(
false.expr(),
0.expr(),
value.expr(),
);
});
// condition이 false면 lookup 건너뜀
// input_rlc = 0 * rlc(...) = 0
// 테이블의 첫 행 (모두 0)과 매칭
Self { condition, value }
}
```

```
}
```

적용 사례:

- 선택적 opcode 처리 (PUSH, CALL 등)
- 조건부 메모리 접근
- 옵션 필드 처리

5.3 Lookup 결과 재사용

한 번 lookup한 결과를 여러 곳에서 재사용합니다.

```
pub struct SharedLookupGadget<F> {
// Lookup 결과를 저장하는 셀
result: Cell<F>,
// 결과가 유효한지 표시
is_valid: Cell<F>,
}


impl<F: Field> SharedLookupGadget<F> {
pub fn configure(cb: &mut ConstraintBuilder<F>) -> Self {
let result = cb.query_cell();
let is_valid = cb.query_cell();
// 첫 번째 사용: lookup 수행
cb.condition(is_valid.expr(), |cb| {
cb.bytecode_lookup(
```

```
cb.curr.state.code_hash.expr(),
cb.curr.state.program_counter.expr(),
1.expr(),
result.expr(),
);
});
Self { result, is_valid }
}
// 이후 사용: result 셀만 참조 (lookup 불필요)
pub fn value(&self) -> Expression<F> {
self.result.expr()
}
}
```

효과:

- Lookup 횟수 감소
- 제약 조건 감소
- 일관성 보장 (같은 값 참조)

5.4 테이블 프리로딩

자주 사용되는 테이블을 미리 로드하여 lookup 성능을 향상시킵니다.

```
// Fixed 테이블은 setup 시 한 번만 로드
pub struct FixedTableLoader<F> {
```

```
tables: Vec<FixedTable<F>>,
}

impl<F: Field> FixedTableLoader<F> {
pub fn load_all(&self, layouter: &mut impl Layouter<F>) -> Result<(), Error>
// 모든 Fixed 테이블을 한 번에 로드
for table in &self.tables {
table.load(layouter)?;
}
Ok(())
}
}

// 사용 시점에는 이미 로드된 테이블 참조
impl<F: Field> Circuit<F> for MyCircuit<F> {
fn synthesize(
&self,
config: Self::Config,
mut layouter: impl Layouter<F>,
) -> Result<(), Error> {
// Setup 단계에서 테이블 로드
config.fixed_table_loader.load_all(&mut layouter)?;
// 이후 lookup은 로드된 테이블 사용
```

```
// ...
}
}
```

장점:

- Setup 시간 최적화
- 메모리 지역성 향상
- 캐시 효율성 증가

6. 성능 측정 및 분석

6.1 Lookup 통계 수집

```
pub struct LookupStats {
pub total_lookups: usize,
pub lookups_by_table: BTreeMap<Table, usize>,
pub lookups_by_phase: BTreeMap<u8, usize>,
pub avg_columns_per_lookup: f64,
}

impl LookupStats {
pub fn collect<F: Field>(circuit: &impl Circuit<F>) -> Self {
let mut stats = Self::default();
// 회로 분석하여 통계 수집
```

```
// ...
stats
}
pub fn report(&self) {
println!("=== Lookup Statistics ===");
println!("Total lookups: {}", self.total_lookups);
println!("\nBy table:");
for (table, count) in &self.lookups_by_table {
println!(" {:?}: {}", table, count);
}
println!("\nBy phase:");
for (phase, count) in &self.lookups_by_phase {
println!(" Phase {}: {}", phase, count);
}
println!("\nAvg columns per lookup: {:.2}", self.avg_columns_per_lookup);
}
}
```

6.2 병목 지점 식별

```
// Lookup 프로파일링
pub fn profile_lookups<F: Field>(circuit: &impl Circuit<F>) {
let start = Instant::now();
```

```rust
// 테이블별 lookup 시간 측정
let mut table_times: BTreeMap<Table, Duration> = BTreeMap::new();
for table in Table::iter() {
let table_start = Instant::now();
// 해당 테이블의 모든 lookup 수행
perform_lookups_for_table(circuit, table);
let table_duration = table_start.elapsed();
table_times.insert(table, table_duration);
}
let total_duration = start.elapsed();
// 결과 출력
println!("=== Lookup Performance ===");
println!("Total time: {:?}", total_duration);
println!("\nTime by table:");
for (table, duration) in &table_times {
let percentage = duration.as_secs_f64() / total_duration.as_secs_f64() * 100.0;
println!(" {:?}: {:?} ({:.1}%)", table, duration, percentage);
}
// 병목 테이블 식별
let bottleneck = table_times
.iter()
.max_by_key(|(_, duration)| *duration)
.unwrap();
```

```
println!("\nBottleneck: {:?}", bottleneck.0);
}
```

6.3 최적화 효과 측정

```
// 최적화 전후 비교
pub struct OptimizationBenchmark {
pub before: BenchmarkResult,
pub after: BenchmarkResult,
}

pub struct BenchmarkResult {
pub proof_time: Duration,
pub proof_size: usize,
pub num_lookups: usize,
pub num_constraints: usize,
pub memory_usage: usize,
}

impl OptimizationBenchmark {
pub fn report(&self) {
println!("=== Optimization Results ===");
let time_improvement =
```

```
(1.0 - self.after.proof_time.as_secs_f64() / self.before.proof_time.as_secs_f64(

println!("Proof time: {:?} → {:?} ({:.1}% faster)",

self.before.proof_time, self.after.proof_time, time_improvement);

let  size_reduction =

(1.0 - self.after.proof_size as f64 / self.before.proof_size as f64) * 100.0;

println!("Proof size: {} → {} bytes ({:.1}% smaller)",

self.before.proof_size, self.after.proof_size, size_reduction);

let  lookup_reduction =

(1.0 - self.after.num_lookups as f64 / self.before.num_lookups as f64) * 100.0;

println!("Lookups: {} → {} ({:.1}% fewer)",

self.before.num_lookups, self.after.num_lookups, lookup_reduction);

let  constraint_reduction =

(1.0 - self.after.num_constraints as f64 / self.before.num_constraints as f64) *

println!("Constraints: {} → {} ({:.1}% fewer)",

self.before.num_constraints, self.after.num_constraints, constraint_reduction);

}

}
```

7. 일반적인 함정과 해결책

함정 1: 과도한 RLC 사용

```
// ❌ 나쁜 예: 2개 컬럼에 RLC 사용
```

```
let input_rlc = rlc::expr(&[col1.expr(), col2.expr()], challenge);
// RLC 계산 비용 > 2개 컬럼 직접 lookup 비용


// ✅ 좋은 예: 2개 컬럼은 직접 lookup
meta.lookup_any("two column lookup", |meta| {
vec![
(col1.expr(), table.col1.expr()),
(col2.expr(), table.col2.expr()),
]
});
```

기준: 3개 이상의 컬럼일 때 RLC 사용 고려

함정 2: 불필요한 Phase 2 컬럼

```
// ❌ 나쁜 예: Challenge를 사용하지 않는데 Phase 2
let unnecessary = meta.advice_column_in(SecondPhase);
meta.create_gate("simple gate", |meta| {
let value = meta.query_advice(unnecessary, Rotation::cur());
vec![value * (value - 1.expr())] // Boolean 제약, challenge 불필요
});


// ✅ 좋은 예: Phase 1 컬럼 사용
let necessary = meta.advice_column(); // FirstPhase
```

```
meta.create_gate("simple gate", |meta| {
let value = meta.query_advice(necessary, Rotation::cur());
vec![value * (value - 1.expr())]
});
```

원칙: Challenge를 사용하는 경우에만 Phase 2 컬럼 사용

함정 3: 테이블 크기 과소평가

```
// ❌ 나쁜 예: 테이블 크기 제한 없음
pub fn load_unlimited_table<F: Field>(
&self,
layouter: &mut impl Layouter<F>,
data: &[Entry],
) -> Result<(), Error> {
// data가 매우 클 수 있음 → 메모리 부족, 증명 시간 폭증
for (offset, entry) in data.iter().enumerate() {
// 할당...
}
Ok(())
}

// ✅ 좋은 예: 테이블 크기 제한
pub fn load_bounded_table<F: Field>(
```

```
&self,
layouter: &mut impl Layouter<F>,
data: &[Entry],
max_rows: usize,
) -> Result<(), Error> {
if data.len() > max_rows {
return Err(Error::Synthesis);
}
for (offset, entry) in data.iter().enumerate() {
// 할당...
}
Ok(())
}
```

권장사항: 모든 테이블에 최대 크기 제한 설정

함정 4: Lookup 순서 무시

```
// ❌ 나쁜 예: 정렬되지 않은 lookup
// RW 테이블은 정렬되어 있는데 lookup은 무작위 순서
for rw in rw_ops.iter() {
cb.rw_lookup(rw); // 캐시 미스 발생 가능
}
```

```
// ✅ 좋은 예: 정렬된 lookup
// RW 테이블과 같은 순서로 lookup
let sorted_rw_ops = rw_ops.sort_by_key(|rw| rw.rw_counter);
for rw in sorted_rw_ops.iter() {
cb.rw_lookup(rw); // 캐시 효율성 향상
}
```

효과: 캐시 적중률 향상, 메모리 접근 패턴 최적화

8. 최적화 체크리스트

테이블 최적화

- [ ] 중복 데이터 제거
- [ ] 효율적인 인코딩 사용
- [ ] 테이블 크기 제한 설정
- [ ] 패딩 최소화
- [ ] 필요한 데이터만 포함

Lookup 최적화

- [ ] 조건부 lookup 사용
- [ ] 불필요한 lookup 제거
- [ ] Lookup 결과 캐싱
- [ ] Batch lookup 적용
- [ ] Selector 기반 활성화

컬럼 최적화

- [ ] RLC로 다중 컬럼 압축
- [ ] 컬럼 재사용 전략
- [ ] Lookup 전용 컬럼 할당
- [ ] 동적 컬럼 할당 고려
- [ ] 컬럼 수 최소화

Phase 최적화

- [ ] 적절한 Phase 선택
- [ ] Phase 2 컬럼 최소화
- [ ] Challenge 재사용
- [ ] Phase별 컬럼 분리
- [ ] 불필요한 Phase 제거

성능 측정

- [ ] Lookup 통계 수집
- [ ] 병목 지점 식별
- [ ] 최적화 효과 측정
- [ ] 메모리 사용량 모니터링
- [ ] 증명 시간 프로파일링

9. 최적화 사례 연구

사례 1: EVM Circuit Lookup 최적화

문제: EVM Circuit에서 과도한 RW lookup으로 인한 성능 저하

해결책:

1. Lookup 전용 컬럼 할당 (8개)
2. 조건부 lookup 적용 (PUSH, CALL 등)
3. RLC를 통한 12개 컬럼 압축

결과:

- Lookup 횟수: 50% 감소
- 증명 시간: 30% 단축
- 회로 크기: 20% 감소

사례 2: Bytecode Table 중복 제거

문제: 같은 컨트랙트 코드가 여러 번 저장되어 테이블 크기 증가

해결책:

1. 코드 해시로 중복 제거
2. BTreeMap을 사용한 고유 바이트코드 관리
3. 언롤링된 바이트만 저장

결과:

- 테이블 크기: 70% 감소 (인기 컨트랙트의 경우)
- 메모리 사용량: 60% 감소
- 증명 크기: 15% 감소

사례 3: Keccak Table RLC 압축

문제: 5개 컬럼 lookup으로 인한 제약 조건 증가

해결책:

1. 5개 컬럼을 RLC로 압축
2. lookup_input challenge 사용
3. Phase 2 컬럼에 RLC 저장

결과:

- 제약 조건: 80% 감소 (5개 → 1개)
- 증명 크기: 10% 감소
- 증명 시간: 약간 증가 (RLC 계산 비용)

10. 모범 사례

10.1 테이블 설계

```
// ✅ 좋은 테이블 설계
pub struct OptimizedTable {
pub q_enable: Column<Fixed>, // 조건부 lookup용
pub key: Column<Advice>, // 최소한의 키 컬럼
pub value: Column<Advice>, // 필요한 값만
}

impl<F: Field> OptimizedTable {
pub fn configure(meta: &mut ConstraintSystem<F>) -> Self {
let q_enable = meta.fixed_column();
let key = meta.advice_column();
let value = meta.advice_column();
```

```
// 컬럼 수 최소화: 3개
Self { q_enable, key, value }
}
pub fn load(
&self,
layouter: &mut impl Layouter<F>,
data: &[Entry],
max_rows: usize,
) -> Result<(), Error> {
// 크기 제한 확인
if data.len() > max_rows {
return Err(Error::Synthesis);
}
layouter.assign_region(
|| "optimized table",
|mut region| {
// 첫 행: 조건부 lookup용 패딩
region.assign_fixed(
|| "q_enable padding",
self.q_enable,
0,
|| Value::known(F::zero()),
)?;
```

```
// 중복 제거된 데이터만 할당
let unique_data: BTreeSet<_> = data.iter().collect();
for (i, entry) in unique_data.iter().enumerate() {
let offset = i + 1;
region.assign_fixed(
|| "q_enable",
self.q_enable,
offset,
|| Value::known(F::one()),
)?;
// 데이터 할당...
}
Ok(())
},
)
}
}
```

10.2 Lookup 구성

```
// ✅ 좋은 lookup 구성
impl<F: Field> ConstraintBuilder<F> {
pub fn optimized_lookup(
```

```rust
&mut self,
name: &str,
condition: Expression<F>,
columns: Vec<Expression<F>>,
table: &impl LookupTable<F>,
) {
// 조건부 lookup
let input_expr = if columns.len() >= 3 {
// 3개 이상: RLC 사용
condition.clone() * rlc::expr(&columns, self.challenges.lookup_input())
} else {
// 2개 이하: 직접 lookup
condition.clone() * columns[0].clone()
};
let table_expr = if columns.len() >= 3 {
rlc::expr(&table.table_exprs(self.meta), self.challenges.lookup_input())
} else {
table.table_exprs(self.meta)[0].clone()
};
self.add_lookup(name, vec![(input_expr, table_expr)]);
}
}
```

10.3 Phase 관리

```
// ✅ 좋은 phase 관리
pub struct PhaseOptimizedConfig<F> {
// Phase 1: 기본 데이터
pub basic_columns: Vec<Column<Advice>>,
// Phase 2: RLC 데이터만
pub rlc_columns: Vec<Column<Advice>>,
// Challenge
pub challenges: Challenges<Expression<F>>,
}

impl<F: Field> PhaseOptimizedConfig<F> {
pub fn configure(
meta: &mut ConstraintSystem<F>,
num_basic: usize,
num_rlc: usize,
) -> Self {
// Phase 1 컬럼: 대부분
let basic_columns = (0..num_basic)
.map(|_| meta.advice_column())
.collect();
// Phase 2 컬럼: 최소한
```

```
let rlc_columns = (0..num_rlc)
.map(|_| meta.advice_column_in(SecondPhase))
.collect();
let challenges = Challenges::construct_p1(meta);
Self {
basic_columns,
rlc_columns,
challenges,
}
}
}
```

10.4 성능 모니터링

```
// ✅ 좋은 성능 모니터링
pub struct LookupMonitor {
start_time: Instant,
lookup_count: usize,
table_sizes: BTreeMap<String, usize>,
}

impl LookupMonitor {
pub fn new() -> Self {
```

```
Self {
start_time: Instant::now(),
lookup_count: 0,
table_sizes: BTreeMap::new(),
}
}
pub fn record_lookup(&mut self, table_name: &str) {
self.lookup_count += 1;
*self.table_sizes.entry(table_name.to_string()).or_insert(0) += 1;
}
pub fn report(&self) {
let elapsed = self.start_time.elapsed();
println!("=== Lookup Monitor Report ===");
println!("Total time: {:?}", elapsed);
println!("Total lookups: {}", self.lookup_count);
println!("Lookups per second: {:.2}",
self.lookup_count as f64 / elapsed.as_secs_f64());
println!("\nLookups by table:");
for (table, count) in &self.table_sizes {
let percentage = *count as f64 / self.lookup_count as f64 * 100.0;
println!(" {}: {} ({:.1}%)", table, count, percentage);
}
}
```

```
}
```

결론

Lookup 최적화는 ZKEVM 회로의 성능을 결정하는 핵심 요소입니다. 효과적인 최적화를 위해서는:

핵심 원칙

1. **테이블 크기 최소화**
   - 중복 제거
   - 효율적인 인코딩
   - 크기 제한 설정
2. **Lookup 횟수 감소**
   - 조건부 lookup
   - Lookup 통합
   - 결과 캐싱
3. **컬럼 수 조정**
   - RLC 압축 (3개 이상)
   - 컬럼 재사용
   - 전용 컬럼 할당
4. **Phase 최적화**
   - 적절한 Phase 선택
   - Phase 2 최소화
   - Challenge 재사용

최적화 효과

적절한 lookup 최적화를 통해:

- **증명 시간**: 20-40% 단축
- **증명 크기**: 10-30% 감소
- **메모리 사용**: 30-60% 감소
- **회로 크기**: 15-25% 감소

지속적인 개선

Lookup 최적화는 일회성 작업이 아닙니다:

- 정기적인 성능 측정
- 병목 지점 식별
- 새로운 최적화 기법 적용
- 트레이드오프 분석

효과적인 lookup 최적화를 통해 ZKEVM의 확장성과 효율성을 크게 향상시킬 수 있습니다.

관련 문서

- 05-테이블-시스템/09-Lookup-메커니즘.md - Lookup의 기본 동작 원리
- 08-최적화-기법/01-CellManager.md - 셀 관리 최적화
- 08-최적화-기법/02-RLC-기법.md - RLC를 통한 데이터 압축
- 02-핵심-회로/01-EVM-Circuit.md - EVM Circuit의 lookup 사용
- 02-핵심-회로/02-State-Circuit.md - State Circuit의 lookup 최적화